Behavioral Synthesis of ICs

Supplementary Examples

Credentials: Arash Ahmadi

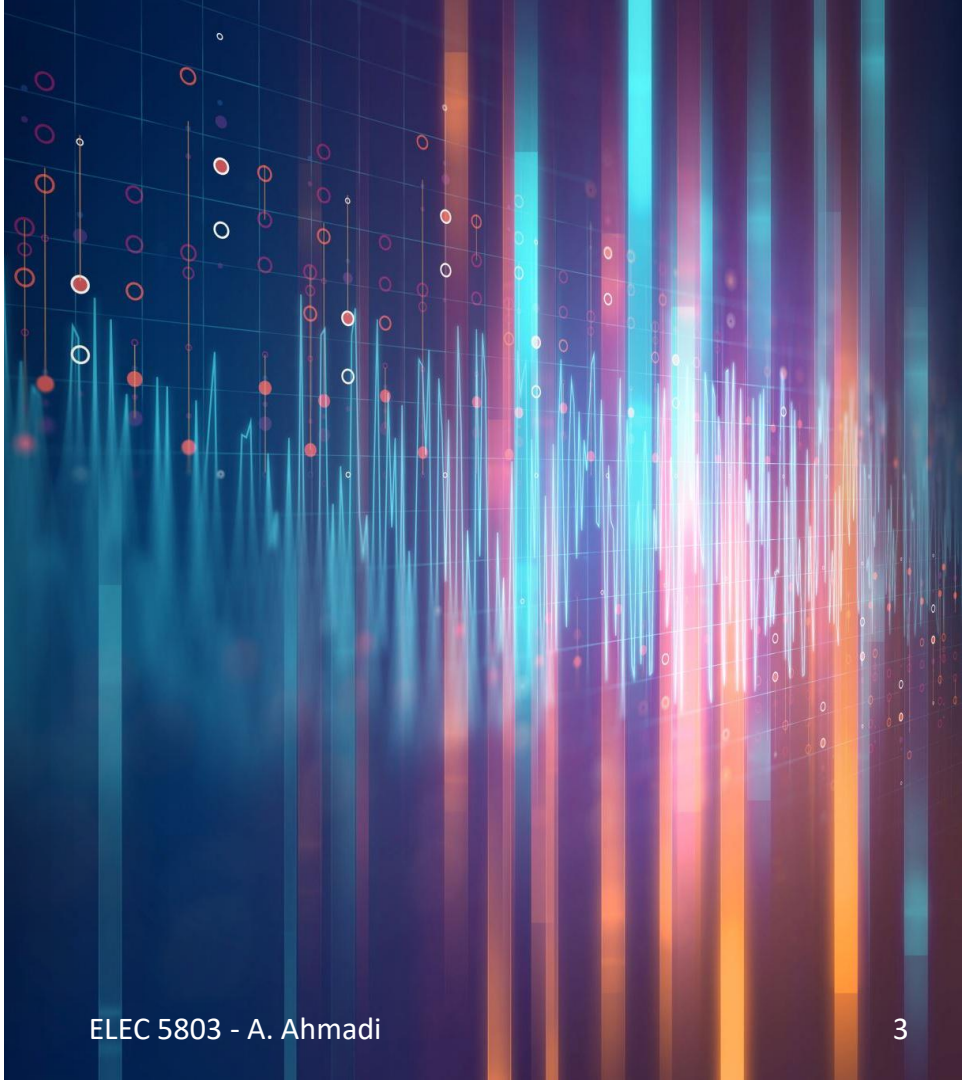
Carleton University

Outline

- DSP Algorithms
  - Filtering
  - Transforms
- Computational Accuracy
  - Examples

DSP Algorithms

- Signal processing algorithms originate from pure mathematical methods to analyze the functions.
- There is a wide variety of DSP algorithms which have been divided into different categories from different points of view, but since algorithms application and mathematical properties are out of this our concern.
- Here, we focus on two broad categories:
  - Filtering
  - Transforms.



DSP Algorithms: Filtering

DSP Algorithms: Filtering

→ In general, a system, and most signal transforms, can be described in the form of a difference equation as illustrated in Equation:

$$F\{x, f(x), f(x+h), f(x+2h), ..., f(x+nh)\} = 0,$$

→ where, F can be a nonlinear time variant function. In the simplest case where F is linear

→ and time invariant, LTI system, the relationship between input, x and output y is:

$$\sum_{k=0}^{N} a_k \cdot y[n-k] = \sum_{k=0}^{M} b_k \cdot x[n-k],$$

DSP Algorithms: Filtering

→ And by using Z transform ($Z\{\}$), we have:

$$\mathbf{Z}\{h[n]\} = H(z) = \frac{\sum_{k=0}^{N} a_k \cdot z^{-k}}{\sum_{k=0}^{M} b_k \cdot z^{-k}}$$

→ where H(z) can be expanded in the form of multiplication of first order fractions to build up a case-cade structure as

$$H(z) = A \cdot \frac{\prod_{k=0}^{M} (z_k - z^{-1})}{\prod_{k=0}^{N} (p_k - z^{-1})}$$

→ or be expanded as addition of second order fractions as

$$H(z) = \sum_{k=0}^{Max\{\frac{N}{2},\frac{M}{2}\}} \frac{b_{2k} + b_{1k} \cdot z^{-1} + b_{0k} \cdot z^{-2}}{a_{2k} + a_{1k} \cdot z^{-1} + a_{0k} \cdot z^{-2}},$$

→ Some very popular examples of these systems are filters.

DSP Algorithms: Filtering

→ From one point of view, filters (systems) can be divided into two categories:

- Finite-Duration Impulse Response (FIR)
- Infinite-Duration Impulse Response (IIR) filters.

→ From the implementation point of view, on the other hand, a FIR filter has no feedback, but an IIR filter has feedback(s) and its computation operation is recursive which increases the complexity and possibility of instability in comparison with FIR system.

→ There are a lot of related works in this regard, specially in design and implementation of filters as the most popular systems in practice; here we review some of the most popular.

DSP Algorithms: Filtering

→ FIR filters and IIR filters are commonly used in DSP algorithms.

→ Assume N and M are finite, $x[n]$ is the input, $y[n]$ is the output, and a_i and b_i are constant coefficients, then every IIR filter has the form

$$y[n] = \sum_{i=0}^{N-1} a_i \cdot x[n-1] - \sum_{i=1}^{M} b_i \cdot y[n-1]$$

→ In a FIR filter, all of the $b_i = 0$.

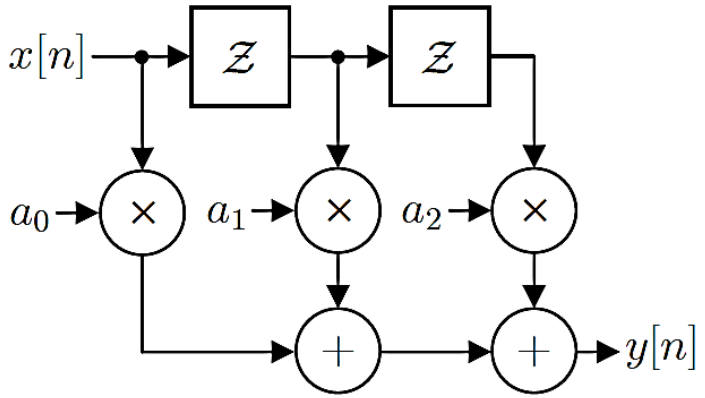
$$y[n] = \sum_{i=0}^{N-1} a_i \cdot x[n-1]$$

→ Two realizations of a FIR example with $N = 3$ are shown in Figure 1. Three separate single constant multiplications are needed in Figure 1a whereas one instance where the same multiplicand is multiplied by three constants is used in Figure 1b.

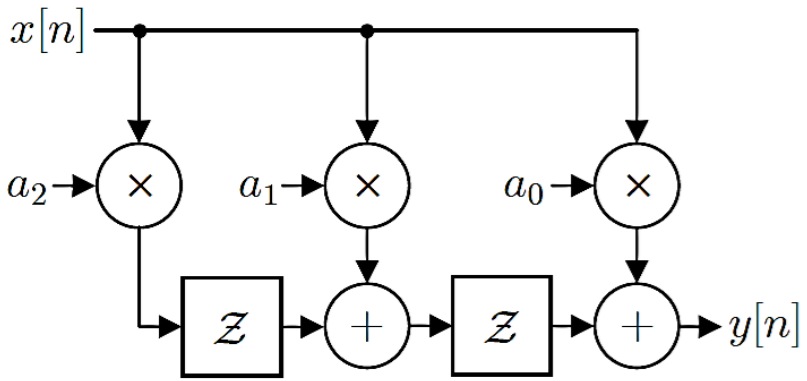
→ **Note**: The delay elements $\boldsymbol{Z}$ have a larger bit width in Figure 1b. Can you explain why?

DSP Algorithms: Filtering

→ **Figure 1**: Two realizations of a FIR filter with $N = 3$ (Z denotes a delay element).



(a) Three separate single constant multiplications.



(b) Three constants that multiply the same multiplicand.

DSP Algorithms: Filtering

→ FIR and IIR filters are arguably the simplest means of performing digital filtering (and therefore likely the cheapest to implement).

→ For **example**, a communication channel may attenuate some frequencies more than others, but by having an estimate of the channel's frequency response, we can apply a filter with the inverse attenuation of each frequency at the output of the channel in order to restore the original signal.

→ In communications, FIR/IIR filters are used for equalization, echo suppression, etc. FIR filters are typically used for up-sampling and down-sampling digital signals. In both cases, an interpolation filter is typically used to smooth the newly sampled signal. This has many applications in audio and video processing (for example, resizing a video or playing audio at different speeds requires a change in the sampling rate).

→ FIR filters are also used in image processing. For **example**, the Laplacian operator can be used for edge detection, and one of the simplest means of implementing this is with a 2-dimensional FIR filter.

DSP Algorithms: Filtering

→ **Example**: consider a FIR filter with coefficient set $W = \{5; 10; 10; 5\}$. The canonical realization of the filter is shown in Figure 2.a which requires using 4 multipliers.

→ If the filter structure is transposed as shown in Figure 2.b, the coefficients $w_0 = 5$ and $w_3 = 5$ can share the same multiplication with 5. Similarly, the coefficients w_1=10 and w_2=10 share the multiplication with 10.

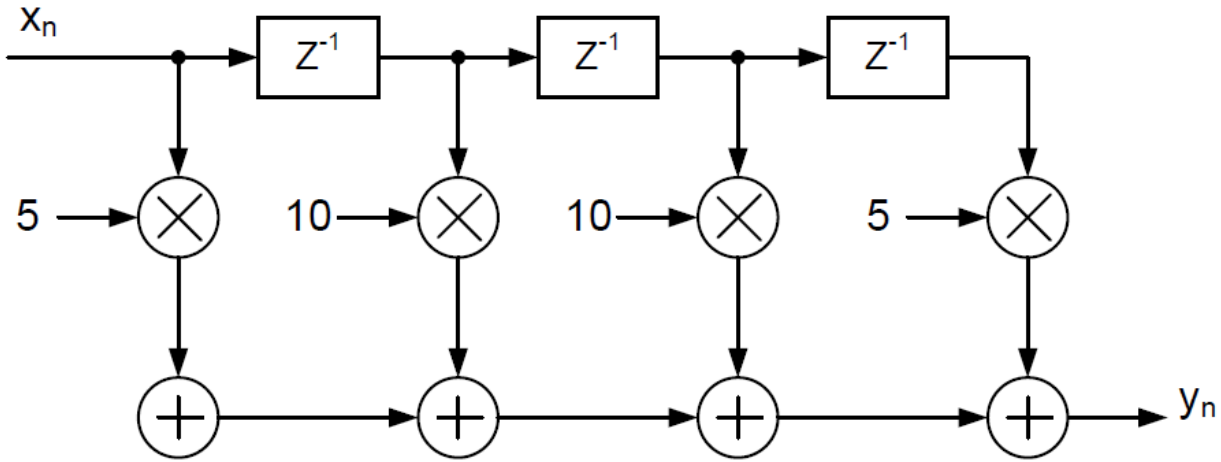
→ The new filter structure after sharing the coefficients is shown in Figure 2.c. In this case, two multipliers are saved.

→ More saving is obtained from noticing that the coefficient 10 can be obtained from shifting 5 to the left by one position. This shift is represented arithmetically by 10 = 5<<1.

→ The power of two relation between the coefficients 5 and 10 reduces the number of multipliers to only one as shown in Figure 2.d. The whole filter operation can be implemented now by just generating the signal $u_n = 5xn$ as shown in Figure 2.d.

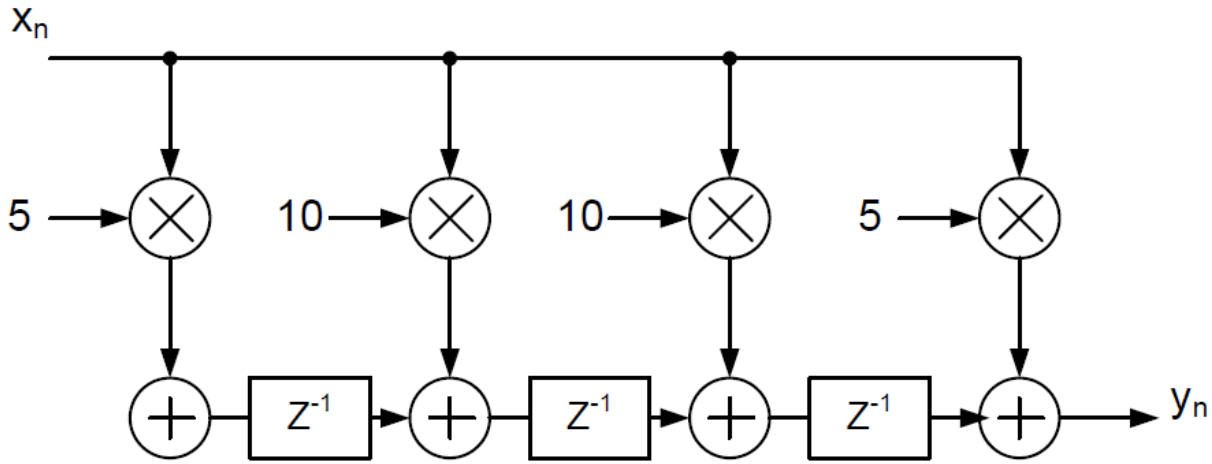
DSP Algorithms: Filtering

→ Figure 2: An example of symmetric FIR filter. (a) The original structure of the FIR filter.



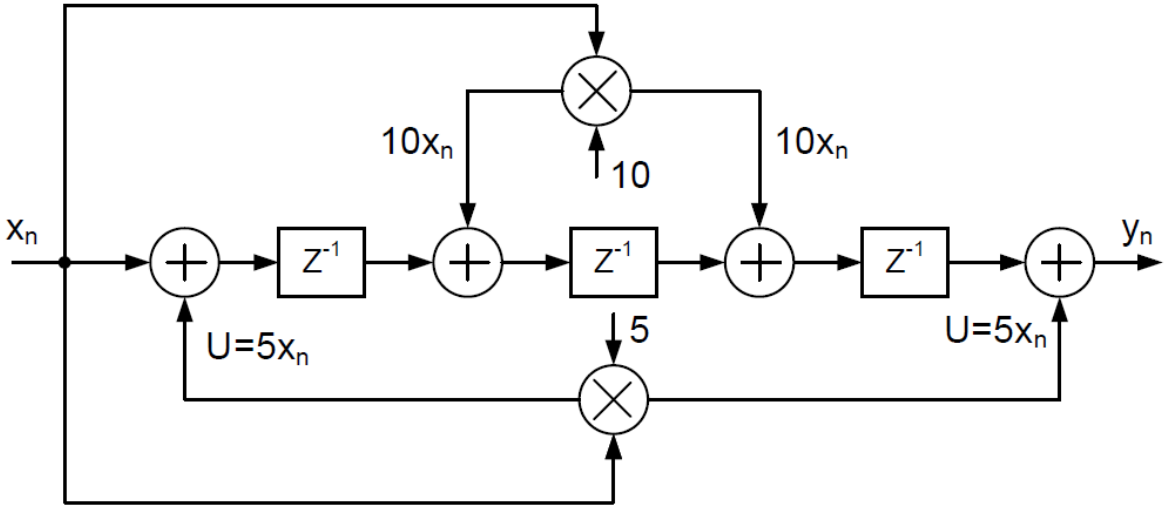
DSP Algorithms: Filtering

→ **Figure 2**: An example of symmetric FIR filter. (b) The transposed structure of the FIR filter.



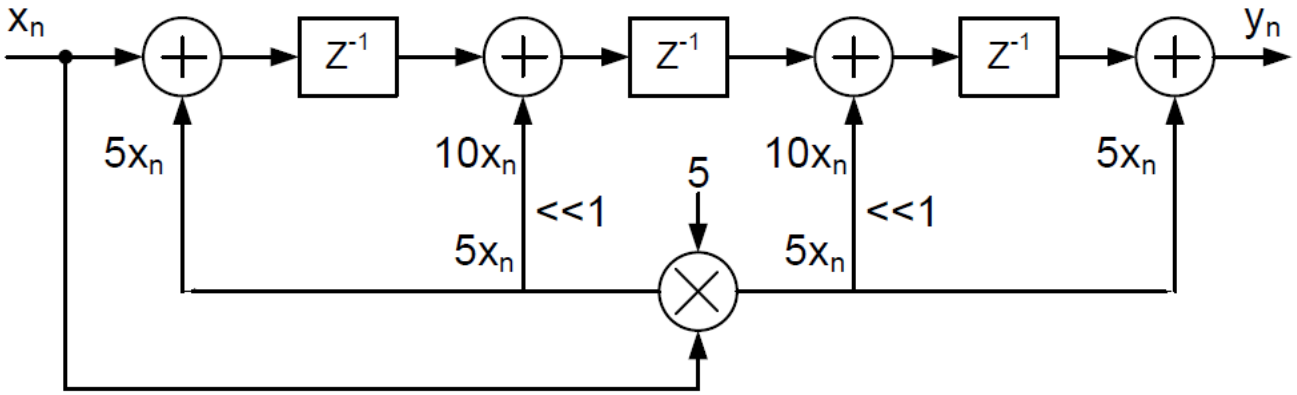
DSP Algorithms: Filtering

→ **Figure 2**: An example of symmetric FIR filter. (c) Sharing the multiplier between the same value coefficients.



DSP Algorithms: Filtering

→ **Figure 2**: An example of symmetric FIR filter. (d) Finding the power of two common factors between the coefficients increases the sharing between them. The symbol $<< i$ means this path shifts the signal to the left by i positions..



DSP Algorithms: Filtering

→ IIR systems have an impulse response with infinite duration in time space. The rational system function that characterizes an IIR system is:

$$y[n] = \sum_{i=0}^{N-1} a_i \cdot x[n-1] - \sum_{i=1}^{M} b_i \cdot y[n-1]$$

→ This also can be expressed as a system's transfer function like:

$$Z\{h[n]\} = H(z) = \frac{\sum_{k=0}^{M} b_k z^{-k}}{1 + \sum_{k=1}^{N} a_k z^{-k}}$$

→ There are several types of realizations structures for IIR filters including:

- Direct Form Structures,
- Cascade Form Structures,
- Parallel Form Structures,
- Lattice Structure,
- Lattice-Ladder Structure.

DSP Algorithms: Filtering

→ There are other structures for IIR systems like: *wave digital structures*, *frequency- response masking* and *all-pass-based forms*, etc. They all represent various tradeoffs; the best choice in a given context is not yet fully understood and may never be.

→ Each method according to its implementation costs has advantages and dis- advantages. For example, parameters like Complexity, Memory Requirements, Speed, Finite Word-length Effects and Flexibility.

→ As an **example**, consider a Two-Pole IIR filter. Three basic structures are shown in Figure 3.

→ By manipulating the general form of an IIR system simply we can reduce the design of a high order system (m,n > 2) to the design of two pole system by which as a building block of the larger system, we can make a more complex systems using them in cascade or parallel form or a mixed structures Figure 4.

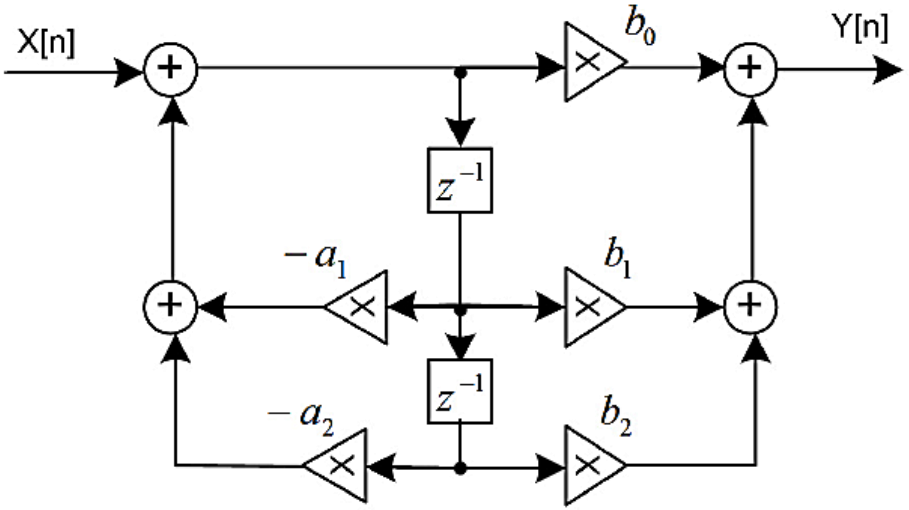
DSP Algorithms: Filtering

→ **Figure 3**: Implementation of general Two-Pole IIR system

a) simple form

b) transposed form

c) lattice form.



(a)

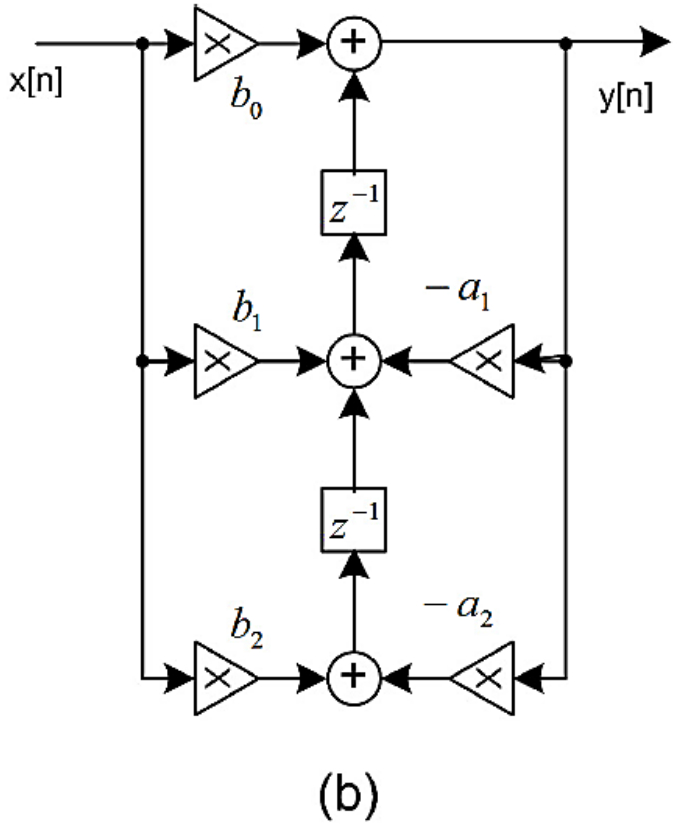
DSP Algorithms: Filtering

→ **Figure 3**: Implementation of general Two-Pole IIR system

a) simple form

b) transposed form

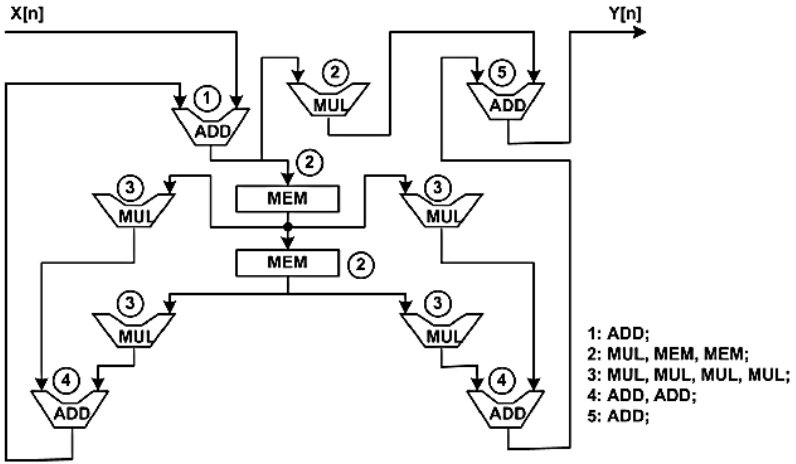
c) lattice form.



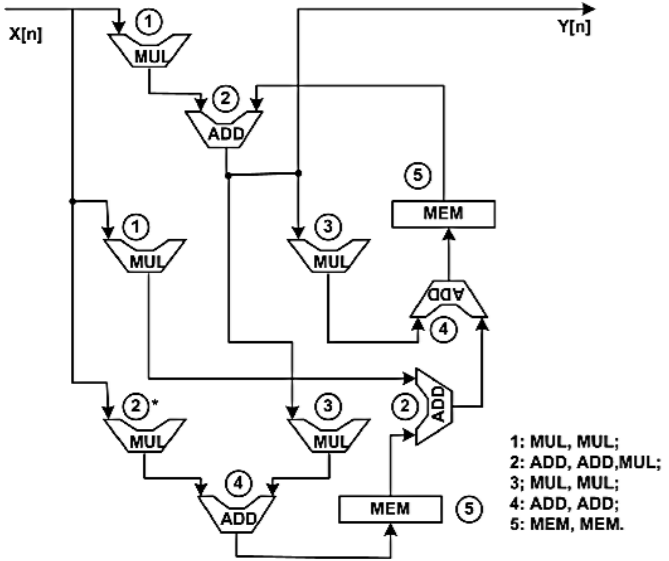
(b)

DSP Algorithms: Filtering

→ **Figure 3***: Hardware realisation of TPF, numbers shows the sequence of operations. a)Simple form b)Transposed form. * This operation can be moved to stage 1 as well.



(a)

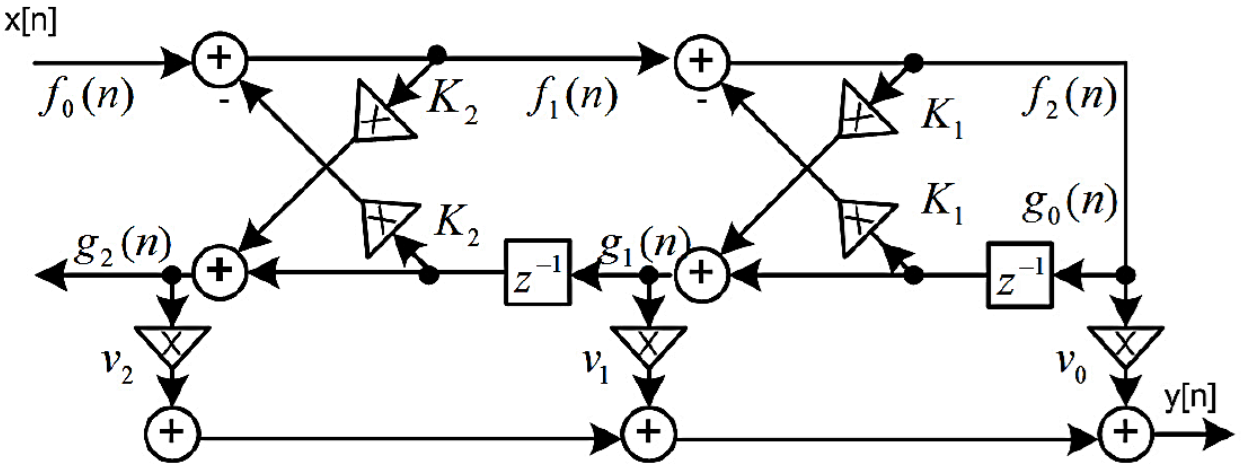


(b)

DSP Algorithms: Filtering

→ **Figure 3**: Implementation of general Two-Pole IIR system

a) simple form
b) transposed form
c) lattice form.



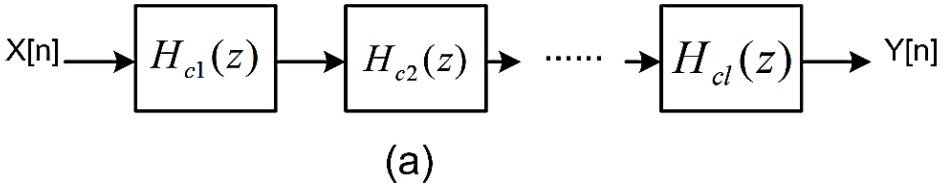
(c)

DSP Algorithms: Filtering

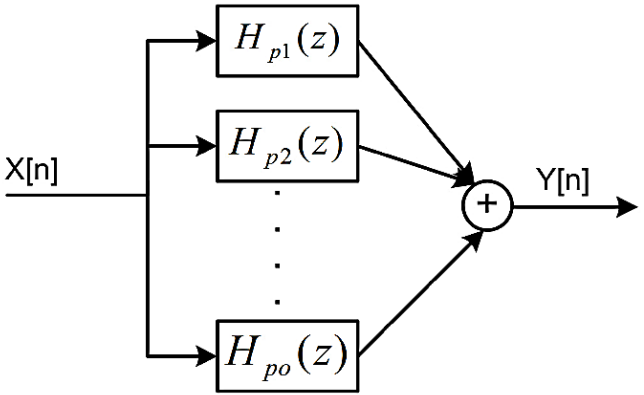
→ **Figure 4**: High order filters can be decomposed in

a) cascade form

b) parallel form.



(a)



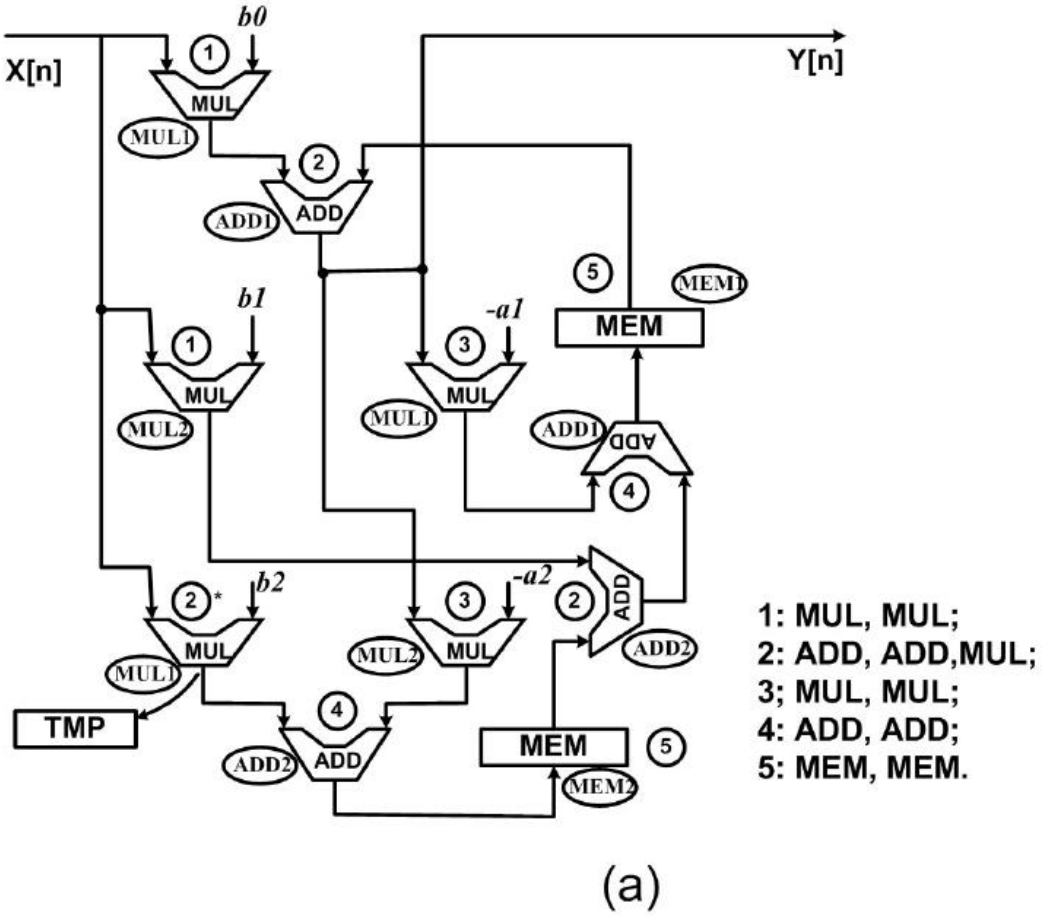
(b)

DSP Algorithms: Filtering

- **Figure** -: Block diagram of two pole system (Transposed type1)

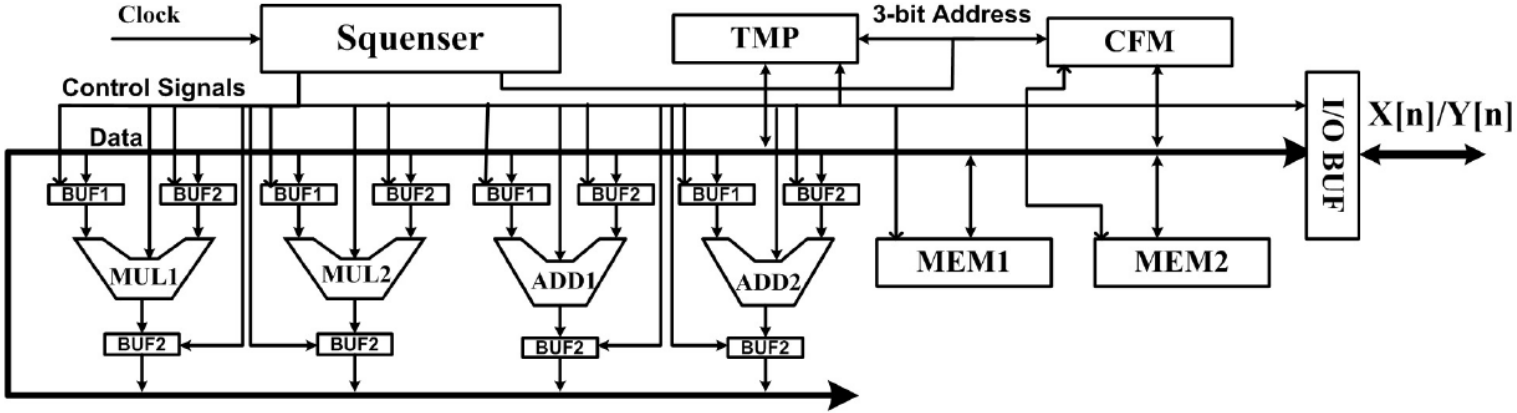
a) operations' sequence

b) Block diagram.



DSP Algorithms: Filtering

- **Figure** -: Block diagram of two pole system (Transposed type1)
  a) operations' sequence
  b) Block diagram.



STEP 1: (X , *b0*) → MUL1, (X , *b1*) → MUL2;
STEP 2: (MUL1 , MEM1) → ADD1, (MUL2 , MEM2) → ADD2, (X , *b2*) → MUL1 → TMP;
STEP 3: (ADD1 , *-a1*) → MUL1, (ADD1 , *-a2*) → MUL2;
STEP 4: (MUL1 , ADD2) → ADD1, (TMP , MUL2) → ADD2;
STEP 5: (ADD2) → MEM2, (ADD1) → MEM1.

(b)

DSP Algorithms: Transforms

Transforms

→ Since Fourier transform of signals has a great impact on signals and systems analysis, many different kind of computation methods have been introduced and discussed for it.

→ According to complexity parameters and resources restrictions, these computation methods have different efficiencies.

→ Here we just discuss the basis of some popular methods for DFT realization and afterward an implementation for each has been designed.

→ There are different algorithms of DFT implementations. Here, as a case study, three basic algorithms are introduced:

- Goertzel Filters,
- Fast Fourier Transform (FFT),
- Matrix Based method.

Transforms

→ A computation procedure that is somewhat more efficient than the direct method is called Goertzel Method (or filters). In this method the DFT can be viewed as the response of a filter [31]. DFT of sequences x[n] is:

$$X[n] = \sum_{k=0}^{N-1} x[k] \cdot W_N^{nk},$$

→ where $W_N^{nk} = e^{-j\frac{2\pi k}{N}}$. Clearly, $W_N^{nN} = e^{-j\frac{2\pi N}{N}} = 1$ so the DFT equation can be rewritten as:

$$\begin{aligned} X[n] &= W_N^{nN} \cdot \sum_{k=0}^{N-1} x[k] \cdot W_N^{nk}, \\ &= \sum_{k=0}^{N-1} x[k] \cdot W_N^{-n(N-k)}, \end{aligned}$$

Transforms

→ System function of the first order Goertzel filter is:

$$H_n(z) = \frac{1}{1 - W_N^{-n} z^{-1}},$$

→ The signal flow graph of a Goertzel filter with unit sample response W_N^{-km} depicted in Figure 5.

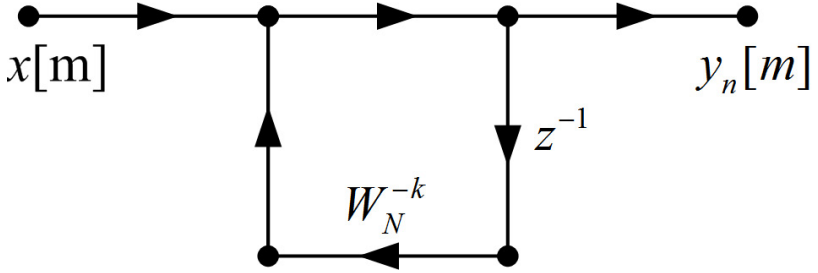
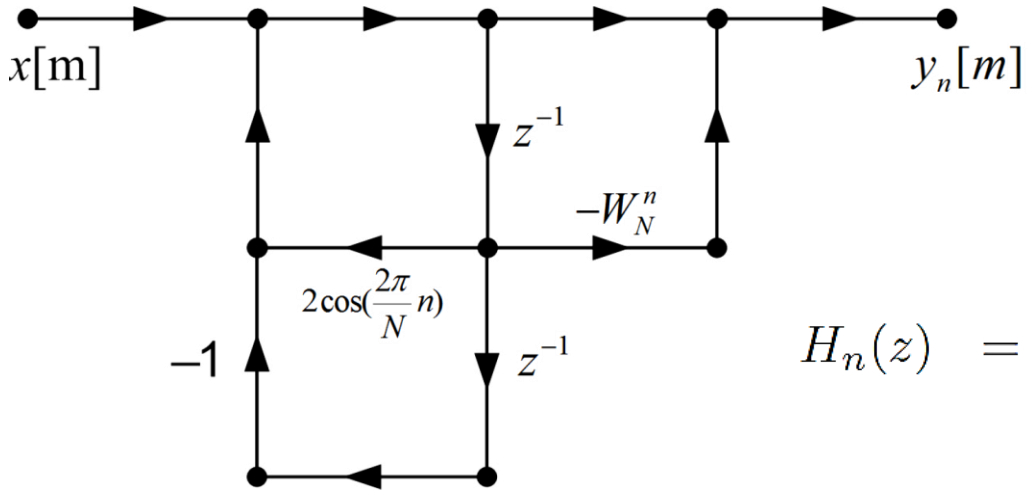


Figure 5: Signal Flow Graph of first order Goertzel filter.

Transforms

→ **Figure 6**: Signal Flow Graph of second order Goertzel filter.



$$H_n(z) = \frac{1 - W_n^N z^{-1}}{(1 - W_N^{-n} z^{-1})(1 - W_n^N z^{-1})},$$

$$= \frac{1 - W_n^N z^{-1}}{1 - 2\cos\left(\frac{2\pi}{N}n\right) z^{-1} + z^{-2}},$$

Transforms

→ The Fast Fourier Transform (FFT) is an important tool used in digital signal processing applications.

→ The definition of the FFT is identical to the DFT, only the method of computation differs.

→ From a matrix based viewpoint, a DFT transformation of an N-point input

$$(x[n]: n = 0, 1, \dots, N-1)$$

→ takes a vector of N coefficients and returns a vector with its values evaluated at N points.

→ Namely this transform is evaluated at the N roots of unity, which are evenly spaced on the unit circle.

→ Since the relationship between input and output is linear, the Fourier transform must be given by a matrix multiplication.

Transforms

→ Discrete Fourier transform given by a matrix multiplication.

$$X[n] = F_N \times x[n],$$

$$\begin{bmatrix} X[0] \\ X[1] \\ X[2] \\ . \\ X[N-1] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & . & 1 \\ 1 & W_N^1 & W_N^2 & . & W_N^{N-1} \\ 1 & W_N^2 & W_N^3 & . & W_N^{2(N-1)} \\ . & . & . & . & . \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & . & W_N^{(N-1)^2} \end{bmatrix} \times \begin{bmatrix} x[0] \\ x[1] \\ x[2] \\ . \\ x[N-1] \end{bmatrix}$$

Transforms

→ In the matrix form, the $W_N = e^{-j\frac{2\pi}{N}}$ is the ith participate of the Nth root of 1, F_N is the NxN Fourier matrix and X[n] is the DFT transformed of x[n].

→ FFT is based on decomposition of the matrix multiplication to smaller matrixes which can affect the computational complexity dramatically.

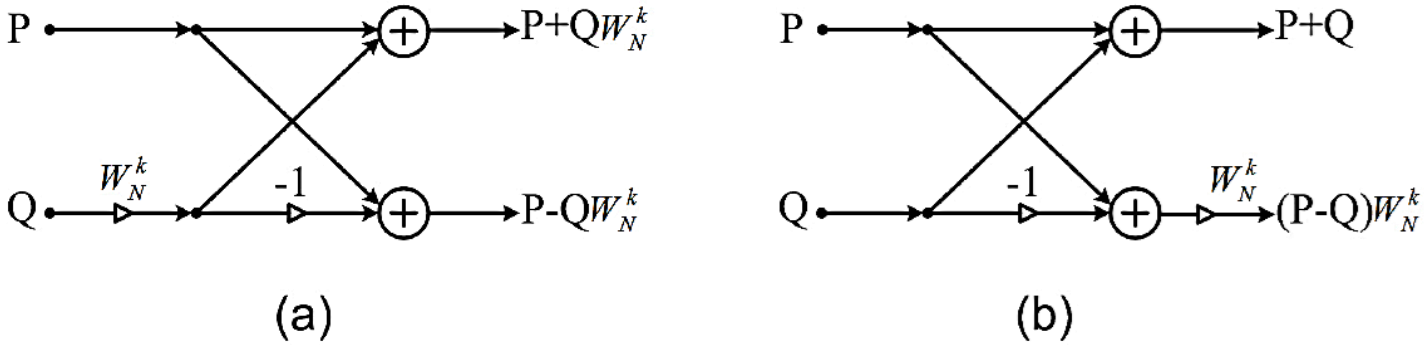
→ To achieve the efficiency of an FFT, it is important that N be a highly composite number, which is typically considered as a power of 2 ($N = 2^M$).

→ Consequently, the whole algorithm breaks down into a repeated application of an elementary transform known in a butterfly structure.

→ Figure 7 shows the general form of radix-2 FFT butter y, more details can be found in DSP textbooks.

Transforms

→ **Figure 7**: Raxid-2 Butterfly Cell for FFT a)DIT Implementation b)DIF Implementation.



Transforms

→ The crucial difficulty of FFT implementation is its inputs and outputs data access and arrangement.

→ For **example**, consider Figure 8 which demonstrates an 8-point FFT implementation in different ways.

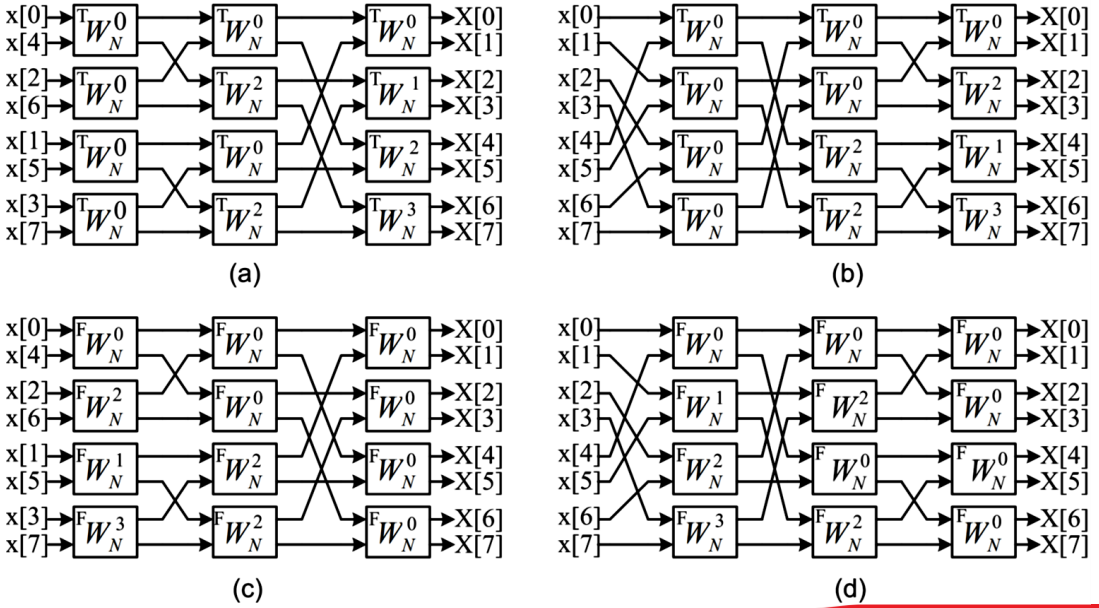
→ Apparently, data passing between butterfly blocks makes an unavoidable computational overhead especially in a single bus systems.

→ Over the years, researchers have developed different forms of FFT for more efficient computation. Radix-2 Butterfly is the smallest element for FFT implementation. The radix of FFT presents the number of inputs that are combined in a butter y.

→ FFT is usually explained around the radix-2 algorithm for simplicity; however, using high order radices saves more computational resources. This saving increases with radix, but there is a very little improvement above radix-4. In radix-4 FFT, each butter y has 4 inputs and 4 outputs, essentially combining two stages of a radix-2 algorithm in one, see Figure 9.

Transforms

→ **Figure 8**: 8-point implementation of FFT by radix-2 Butter y cells a)DIT bit- reversed ordered inputs b)DIT normal ordered inputs c)DIF bit-reversed ordered inputs d)DIF normal ordered inputs (each block is a radix-2 butter y cell)

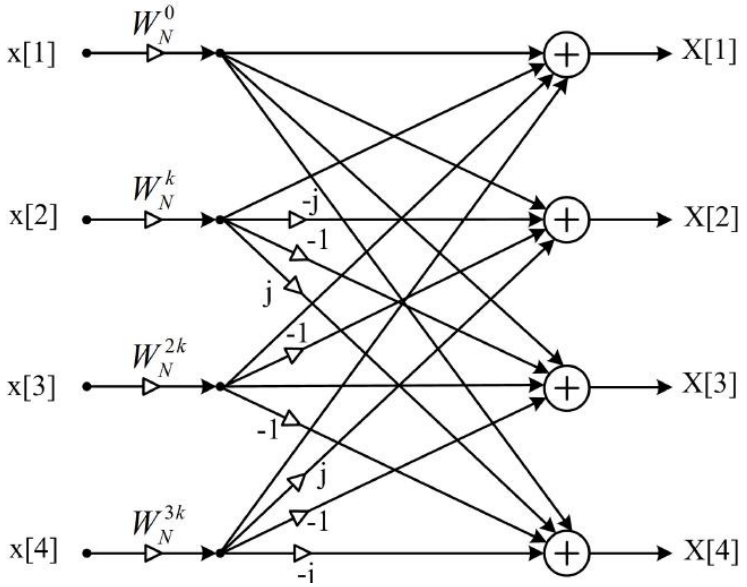


Transforms

→ **Figure 9**: Radix-4 Butter y Cell for FFT implementation.

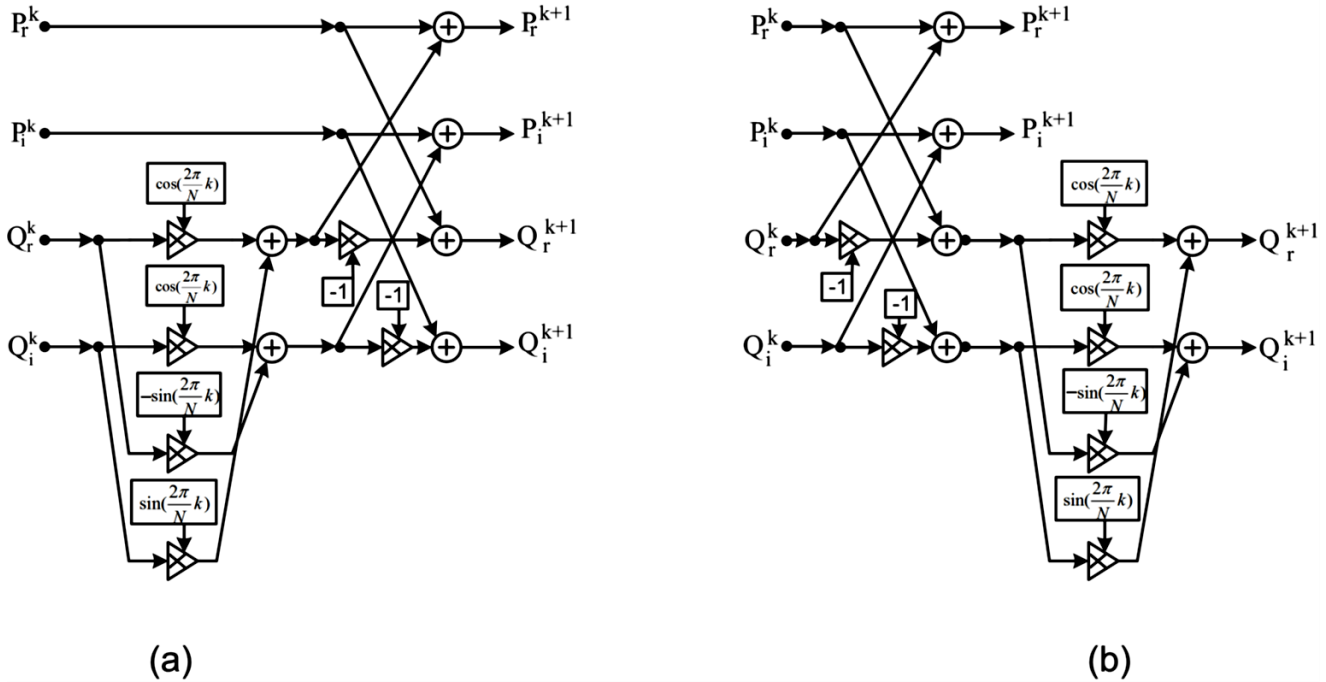
→ The matrix equation for a radix-4 FFT cell

$$\begin{bmatrix} X[1] \\ X[2] \\ X[3] \\ X[4] \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -j & -1 & j \\ 1 & -1 & 1 & -1 \\ 1 & j & -1 & -j \end{bmatrix} \times \begin{bmatrix} x[1] \\ x[2] \\ x[3] \\ x[4] \end{bmatrix}$$



Transforms

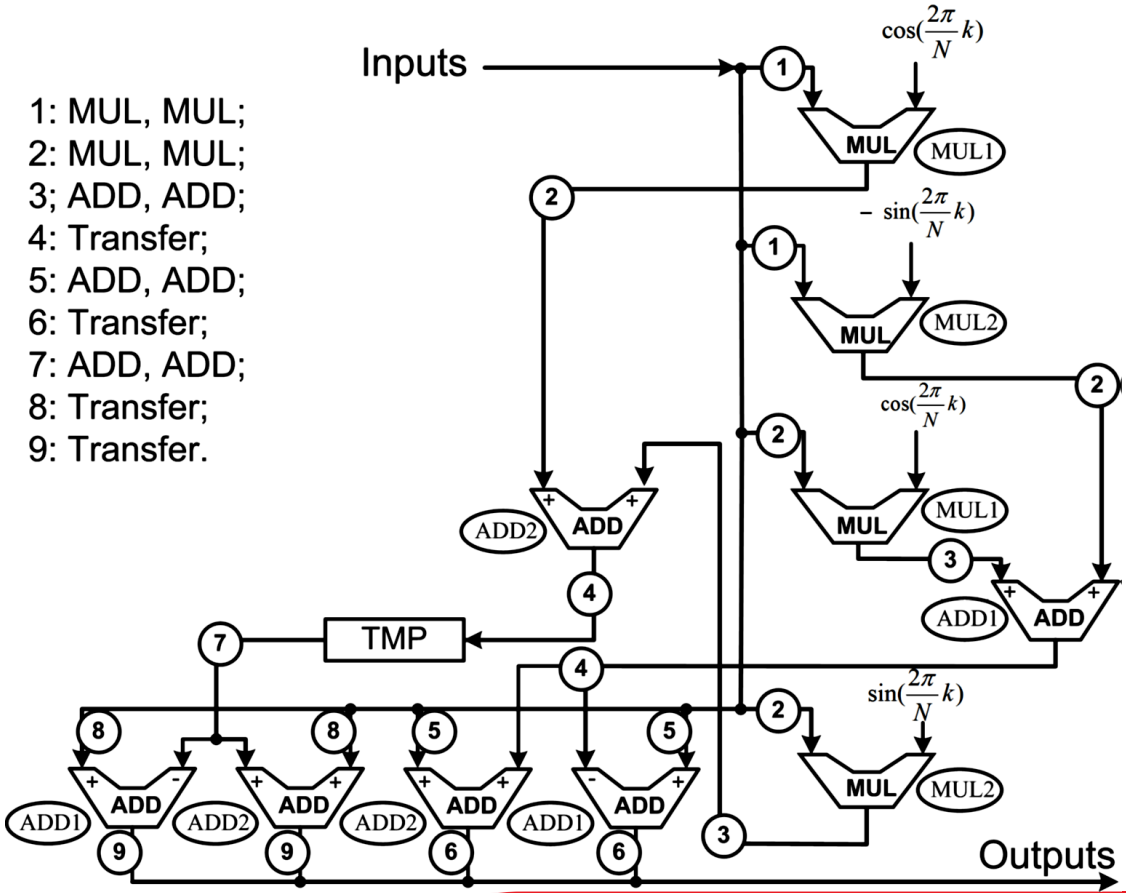
→ **Figure 10**: Radix-2 FFT Butter y Cell block diagram a) DIT form b) DIF form.

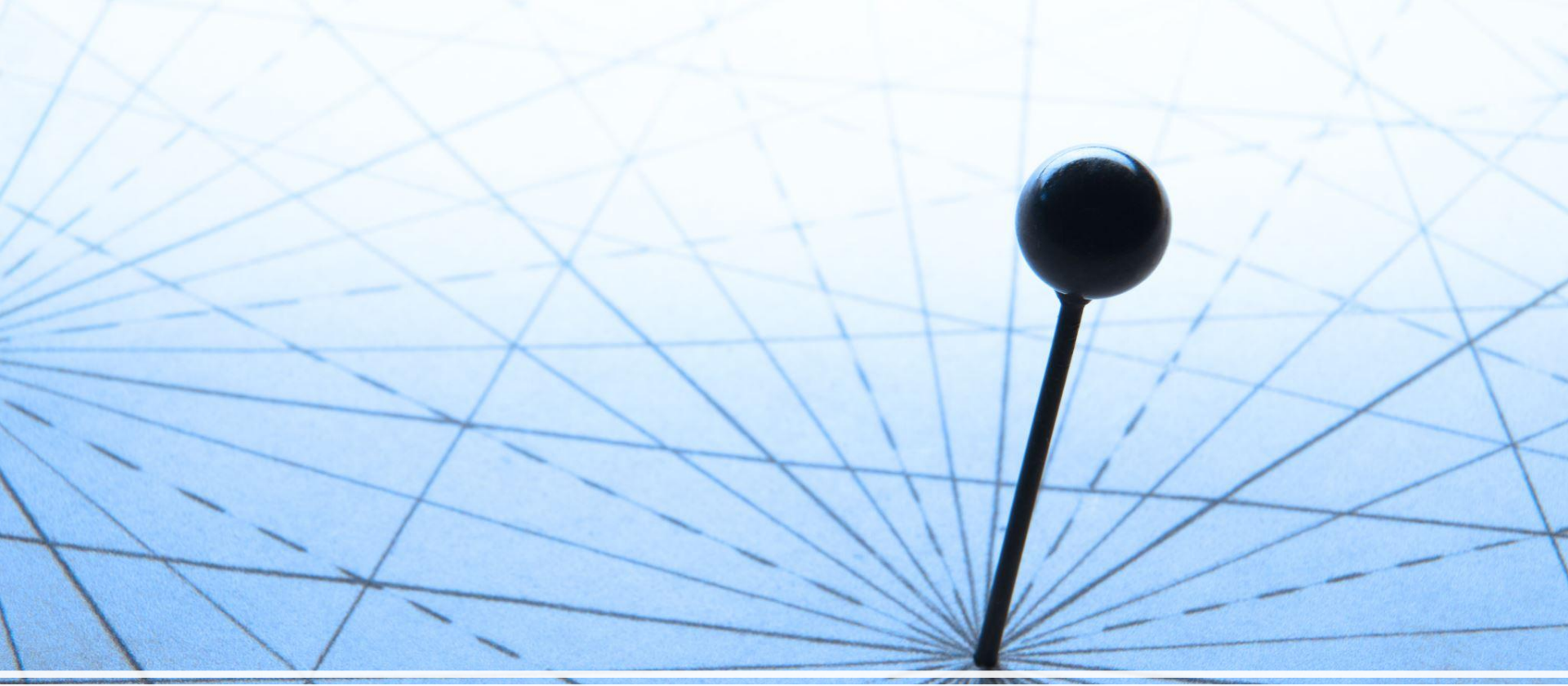


Transforms

→ **Figure 11**: Scheduling diagram for radix-2 butterfly DIT form.

1: MUL, MUL;
2: MUL, MUL;
3; ADD, ADD;
4: Transfer;
5: ADD, ADD;
6: Transfer;
7: ADD, ADD;
8: Transfer;
9: Transfer.





Computational Accuracy

Computational Accuracy

→ **Example**: In a digital signal processing application, you are implementing a filter that computes an output (Y) based on four inputs (A, B, C, and D) using the following equation:

$$[\, Y \;=\; (A \;\times B) \;-\; (C \;\times D)\,]$$

→ The inputs A, B, C, and D are sensor readings that are represented as signed integers. Each of these inputs can vary in the following ranges:

(A) is in the range of [-256, 256]

(B) is in the range of [-128, 128]

(C) is in the range of [-64, 64]

(D) is in the range of [-32, 32]

→ Using interval arithmetic, compute the minimum bit-width required to represent the output Y without any risk of overflow. Remember, the bit-width must account for the sign bit as well.

Computational Accuracy

1. **Maximum and Minimum of Products:**
    1. (A . B) has a range of ([-32768, 32768]).
    2. (C . D) has a range of ([-2048, 2048]).
2. **Range of Y:**
    1. Maximum Y: (32768 - (-2048) = 34816).
    2. Minimum Y: (-32768 - 2048 = -34816).
3. **Bit-width Calculation:** The range of Y is ([-34816, 34816]). To represent this number including the sign, we need one bit for sign and additional bits for the maximum absolute value which is 34816. The number 34816 in binary is approximately represented in 16 bits (without sign bit). Hence including the sign bit, you need **17 bits** to represent Y without overflow.

Computational Accuracy

→ **Example**: Consider implementing a simple moving average filter (also known as a box filter) in an embedded system for smoothing sensor data. The moving average algorithm calculates the average of the last (N) sensor readings, where (N) is the size of the window for the moving average. The algorithm is designed to process a stream of sensor readings in a loop.

→ Given:

- Each sensor reading is a 10-bit signed integer, representing values in the range of ([-512, 511]).
- The moving average window size ((N)) is 16.

→ Using interval arithmetic, compute the minimum bit-width required to represent the output of the moving average filter without any risk of overflow. This computation must account for accumulative operations within the loop for summing sensor readings and the division operation for averaging.

Computational Accuracy

→ **Solving Approach**

1. **Determine the Maximum and Minimum Sums:**
   - Calculate the potential maximum and minimum sum of the (N) sensor readings within the moving average window.
2. **Calculate the Result of the Moving Average:**
   - Find the maximum and minimum possible outputs of the moving average operation.
3. **Calculate the Bit-width:**
   - Determine the range of possible outputs and calculate the required bit-width to represent this range, including the sign bit.

Computational Accuracy

→ **Solution Steps**

1. **Maximum and Minimum of Sum:**
   - With each reading ranging from ([-512, 511]), the maximum sum for 16 readings is (511 × 16 = 8176), and the minimum sum is ((-512) × 16 = -8192).
2. **Range of Moving Average Output:**
   - The moving average divides the sum by (N), so the maximum output is (8176 / 16 = 511) and the minimum output is ((-8192) / 16 = -512).
3. **Bit-width Calculation:**
   - The range for the moving average filter output is ([-512, 511]). This matches the input range, which is already represented by a 10-bit signed integer. However, for the accumulative sum, analyze the requirement:
     - The sum ranges from (-8192) to (8176), which exceeds the original sensor reading's range.
     - To represent (-8192), we need 14 bits (13 for value and 1 for sign), as (2^{13} = 8192).

Computational Accuracy

→ **Example**: Implement a single-pole IIR filter in a digital signal processing system. The IIR filter applies a simple recursive relation between its input and output to achieve an exponential smoothing effect on the incoming signal. The formula for the filter is given as:

$$y[n] = \alpha x[n] + (1 - \alpha) y[n - 1]$$

→ where:

- (y[n]) is the filtered (output) signal at the discrete time (n).
- (x[n]) is the input signal at the discrete time (n).
- (y[n-1]) is the previous output at the discrete time (n-1).
- (\alpha) is the smoothing factor, a constant within the range (0 < \alpha < 1).

→ Assume the following:

- Each input signal ((x[n])) is a 16-bit signed integer, representing values in the range of ([-32768, 32767]).
- The smoothing factor (\alpha) is set to 0.5 for simplicity.
- The initial condition for (y[-1]) (the first output before any inputs are processed) is 0.

→ Using **interval arithmetic**, compute the minimum bit-width required to represent the output of the IIR filter ((y[n])) without any risk of overflow. Consider the accumulative nature of the feedback loop in the filter.

Computational Accuracy

→ **Solving Approach**

1. **Analyze the Output Range:**
    1. Determine the potential maximum and minimum values for the output ((y[n])) considering the feedback loop.
2. **Calculate the Bit-width:**
    1. Based on the range of output values, calculate the required bit-width for representing (y[n]) safely, including the sign bit.

Computational Accuracy

→ **Solution Steps**

1. **Feedback Loop Analysis:**
    1. With (\alpha = 0.5), the equation becomes (y[n] = 0.5 \cdot x[n] + 0.5 \cdot y[n-1]).
    2. Given the max input value, (x[n] = 32767), and substituting in the worst case where (y[n-1] = y[n]) (steadystate), we can solve for (y[n]) in terms of (x[n]) purely.
        1. At steady state, assuming maximum input perpetually, (y[n] = 0.5 \cdot 32767 + 0.5 \cdot y[n]).
        2. Solving for (y[n]) yields the same range as (x[n]), since the contribution from (y[n-1]) in absolute terms cannot exceed the max/min input value due to the multiplication by (0.5) effectively reducing it.
    3. Therefore, the maximum and minimum potential values for (y[n]) are theoretically the same as for (x[n]), due to the dampening effect of the filter.
2. **Bit-width Calculation:**
    1. Given the input signal range is ([-32768, 32767]) and considering the feedback dampening, the maximum range needed for (y[n]) doesn't exceed that of the input signal.
    2. A 16-bit signed integer is sufficient to represent (y[n]) without risk of overflow, taking into account that (y[n]) contains a fractional component and that the calculation is iterative.

Computational Accuracy

→ **Example**: In the implementation of a digital low-pass filter, computational noise arises due to the quantization of coefficients and arithmetic operations. Consider a simple first-order Infinite Impulse Response (IIR) digital low-pass filter defined by the following difference equation:

$$y[n] = \alpha x[n] + (1 - \alpha) y[n - 1]$$

→ where:

- (y[n]) is the filtered output at time step (n).
- (x[n]) is the input signal at time step (n).
- (y[n-1]) is the previous filtered output.
- α is the filter coefficient with a value(\frac{1}{4}) representing the weight given to the current input in the smoothing process.

→ Given the input signal (x[n]) is a stream of 12-bit signed integers, with values in the range from ([-2048, 2047]), and assuming the filter operates using fixed-point arithmetic with a word length of 12-bits for representing both the input and output signals, calculate:

- The maximum computational noise generated due to quantization of the filter coefficient.
- The cumulative effect of computational noise on the filter output after processing 1000 samples, assuming an initial condition of (y[-1] = 0) and that each arithmetic operation introduces an error of (\pm 0.5) LSB (Least Significant Bit) due to quantization.

Computational Accuracy

→ **Approach to Solution**

1. **Quantization of the Filter Coefficient**: Quantization error occurs when converting the continuous range of (α) into discrete levels. With ($\alpha = \frac{1}{4}$) and using 12-bit fixed-point representation, we'll need to approximate it.
2. **Computational Noise per Operation**:
    1. For addition and multiplication operations, compute the noise introduced based on the quantization error and the (±0.5) LSB assumption for each operation.
3. **Cumulative Effect of Computational Noise**:
    1. Deduce how this noise accumulates across 1000 input samples, considering the recursive nature of the filter.

Computational Accuracy

→ **Solution Steps**

1. **Quantization of (\alpha)**:
    1. As (\alpha = \frac{1}{4}) exactly represents the filter coefficient in a fixed-point arithmetic setup, the maximum computational noise from the coefficient quantization itself is zero since (\frac{1}{4}) can be precisely represented in binary as (0.01) (in fractional binary).
2. **Computational Noise per Operation**:
    1. For each multiplication operation ((\alpha \cdot x[n]) and ((1 - \alpha) \cdot y[n-1])), assume error introduced is (\pm 0.5) LSB.
    2. For the addition operation ((\alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1])), assume another (\pm 0.5) LSB error is introduced.
    3. Therefore, for each (n), a maximum of (\pm 1.5) LSB of computational noise could be introduced.
3. **Cumulative Effect**:
    1. If each input sample processing introduces a maximum of (\pm 1.5) LSB of noise, processing 1000 samples could, in the worst case, introduce a maximum cumulative noise of (\pm 1500) LSB in total.
    2. This simplified analysis does not precisely predict the noise because it assumes worst-case addition of errors. Statistically, errors might cancel out or accumulate less aggressively.

Computational Accuracy

→ **Example:** Consider the implementation of a signal processing algorithm that applies a gain factor to a signal through a loop operation. The algorithm processes an input signal (x[n]), where (n = 0, 1, 2, \ldots, N-1), by multiplying each sample by a fixed gain (G) and summing a constant offset (C), under the following operation:

→ [y[n] = G \cdot x[n] + C]

→ The operation is performed in a loop for each sample of the signal.

→ Given:

- The input signal (x[n]) comprises 16-bit signed integers ranging from ([-32768, 32767]).
- The fixed gain (G) is 1.5, which must be quantized for processing.
- The constant offset (C) is 100, to be added post-multiplication by the gain.
- The system processes the algorithm using fixed-point arithmetic with a word length of 16 bits for output signal (y[n]) representation.

→ Calculate:

- The quantization error introduced by representing the gain (G) in the system.
- The computational noise generated after processing (N = 1000) samples, assuming each multiplication and addition operation introduces an error of (\pm 0.5) LSB (Least Significant Bit) due to quantization.

Computational Accuracy

→ **Approach to Solution**

1. **Quantizing the Gain (G)**: Determine the representation error for the gain (G) when quantized for fixed-point arithmetic.
2. **Computational Noise per Operation**: Compute the noise introduced by the quantization error per operation (multiplication by (G) and addition of (C)) and the inherent (\pm 0.5) LSB error assumption for arithmetic operations.
3. **Cumulative Computational Noise**: Estimate how computational noise accumulates across the (N = 1000) samples, considering the loop's repetitive nature.

Computational Accuracy

→ **Solution Steps**

1. **Quantizing the Gain (G)**:
    1. Assuming a 16-bit fixed-point representation, (G = 1.5) cannot be precisely represented. The closest representation may be approximated as (G_{q} \approx 1.499), introducing a quantization error for (G). The exact quantization error would depend on the fractional bit resolution used for (G).
    2. Quantization error for (G), (E_G = |1.5 - G_{q}|).
2. **Computational Noise per Operation**:
    1. Multiplication operation by (G): Given the approximation of (G), each operation introduces a computational noise from the (\pm 0.5) LSB error, plus the error due to (G)'s quantization.
    2. Addition of (C): Introduces a (\pm 0.5) LSB error.
    3. Total operation noise per sample: (E_{total} = E_G + E_{mult} + E_{add}), where (E_{mult}) and (E_{add}) represent the multiplication and addition errors respectively, assumed to be (\pm 0.5) LSB each.
3. **Cumulative Computational Noise**:
    1. The cumulative effect over (N = 1000) samples does not simply sum because computational noise involves random errors that can partly cancel out or compound non-linearly.
    2. A worse-case scenario, neglecting cancellation effects, could suggest a cumulative noise of (1000 \times (E_{total})), but realistically, statistical analysis would provide a more accurate estimate of cumulative computational noise.